LANvas: technical documentation

ProductContext.md

> Working name: **LANvas** (rename freely). Single source of truth for *what we are building and why*. See also: `Architecture.md` (how), `Design.md` (look and feel), `Agents.md` (rules for AI/code contributors and the phase plan).

---

1. One-line pitch

A **Figma-style, offline, LAN-only collaborative design whiteboard** where the room creator controls exactly who can view or edit, boards on the network show up by themselves, nothing is lost on a crash, and the network protocol itself is visible and demonstrable.

2. Context

- **Course:** Computer Networks (CN) Lab project, LAN-Based Multi-User Collaborative Whiteboard.
- **Submission:** fixed by the course (see the brief); no extensions. In-person demo with the internet disabled.
- **How we work:** the project is built **phase by phase** (`Agents.md` §11). A phase starts only after the previous phase is finished **and** its tests pass; the cycle repeats until the whole project is complete and tested. There are no internal calendar milestones.
- **Constraints from the brief:** server-based, client-server over TCP/IP sockets, LAN only, no internet; low-level socket programming is mandatory; Python recommended; standalone server and client executables; PDF docs, PPT, per-member video.
- **How we are graded:** every team has the same core. Marks go to **completeness, correctness, and extra features with effective integration**. Our extras must feel like one coherent product, not a pile of add-ons.

3. Problem and users

User	Need	How we serve it
Student groups in a lab or hostel	Brainstorm and sketch together without internet; talk privately while working	Live board discovery, fast join, room chat and private messages
Instructor / host	Control who is in the room, protect the board, never lose work	Owner-approved access, roles, kick, lock; Ctrl+S checkpoints; crash recovery
Evaluator (grader)	See networking concepts working, quickly	Packet Inspector, chaos simulator, clean docs

4. Product principles

1. **Offline-first, LAN-only.** Nothing ever touches the internet.
2. **Creator is in control.** Knowing the room code or being on the same Wi-Fi is *not* authorization.
3. **The board is always consistent.** Server-authoritative ordering; any client can be dropped and recover.
4. **Nothing is lost.** Every write is atomic and checksummed; a crash of the client or server recovers to the last good state.
5. **The protocol is a feature.** We expose it, we do not hide it.
6. **Figma-lite, done well** beats Figma-everything done badly.

5. Differentiators (our "unique angle")

1. **Creator-gated access with a request-to-join flow** (Section 7).

2. **Figma-style object canvas**: shapes, frames, layers, properties panel, multiplayer cursors, comments.
3. **Live board discovery**: a device that joins the network sees the available boards in its launcher within seconds (only board names are advertised, never content or codes).
4. **Room chat plus private messages**, both kept as history across reconnects and server restarts.
5. **Secure save and recovery**: Ctrl+S makes a verified server checkpoint plus a signed local copy; crash-safe files and an automatic recovery dialog.
6. **Export anywhere**: PNG, JPEG, SVG, PDF, and the native signed `.lanvas` file.
7. **Live Packet Inspector** showing every frame, sequence number, size, and latency.
8. **Chaos / Network Simulator** (delay, duplicate, reorder, drop connection) to prove ordering and recovery live.
9. **Version history** built on the append-only operation log and checkpoints.

6. Feature scope (tiered)

Tiers describe *what*; the phases in `Agents.md` §11 describe *in which order* it is built and tested.

Tier 1: mandatory core (Phases 0 to 4)

- Pen (freehand), eraser, color selection on a shared canvas
- Rectangle, ellipse, line, text (these also serve the "extended" credit)
- Real-time sync, consistent shared state, server-side validation and ordering
- Host a room; **live list of available boards** on the network; join from the list or with LAN address plus room code; participant list with join/leave notices
- Usernames; per-user color; live cursors with name labels
- **Group chat with sender and timestamp, plus private messages between members; chat history retained**
- New joiner receives the current board and recent chat; reconnect recovers the latest state
- Undo / redo with defined multi-user behavior
- Clear canvas (permission required)
- **Save with Ctrl+S** (server checkpoint + signed local copy), open/load `.lanvas`, **crash recovery** on server and client
- **Export:** PNG and `.lanvas`
- **Access control: owner-approved membership** (this is Tier 1 for us, not optional)

Tier 2: the Figma feel (Phase 5)

- Select, move, resize, rotate, multi-select; z-order; layers panel; properties panel
- Zoom, pan, snap guides, align tools, duplicate, group
- Frames, sticky notes, arrows, image insertion
- Comments pinned to the canvas
- **Export:** JPEG, SVG, PDF (including "each frame as a page"), scope = board / selection / frame; password-protected `.lanvas`
- Checkpoint browser and restore (owner); create a room from a `.lanvas` file
- Packet Inspector; chaos simulator

Tier 3: stretch (Phase 6, only if Tiers 1 and 2 pass multi-machine tests)

- Version history timeline slider, presentation (follow-me) mode
- TLS transport, per-page permissions, dark mode polish

Rule: Tier 3 work does not begin until Tier 1 passes a 4-machine LAN test.

Signature tier: "our style" (Phase 6, LLM-accelerated scope, only on a green Tier 1 + Tier 2)

Identity: **a Figma-class canvas that is network-transparent and security-native.**

- **Offline AI assistant (local LLM via Ollama on the LAN host):** "draw a login-flow flowchart", "tidy these notes", "summarize chat and board". The assistant joins as a visible participant with its own role, and its ops pass through the *same* server validation and ACL as any user. It never reads DMs. No internet; optional and fails gracefully if no model is present.
- **Smart connectors:** lines/arrows bound to shape anchors that follow when shapes move (flowcharts, mind maps).
- **Audit log panel (owner):** who did what, when, from which IP/fingerprint; filterable; exportable.
- **Local-first offline edits:** keep editing while disconnected, then merge on reconnect via the op log.
- **Branchable version history:** scrub the timeline, fork a version into a new room.
- **Lite components and auto-layout:** reusable symbols, row/column auto layout for frames.
- **Presentation / follow-me mode:** owner drives everyone's viewport.

7. Access control (key requirement)

Statement: Only people authorized by the room creator can view or edit the board, or read its chat, even if they are on the same network. Unauthorized people can *request* permission.

Roles: `owner` (creator), `editor`, `viewer`, and no access.

User stories

- As an owner, I create a room and become its only initial member.
- As a visitor, I open the app and see the boards available on my network; if I pick one (or connect with a room code) but am not authorized, I see only a "Request access" screen. I receive **no** board data, chat, or participant list.
- As an owner, I get a prompt "Alice (192.168.1.23, fingerprint `7F3A-91C2`) requests access", and I choose Approve as Editor, Approve as Viewer, or Deny.
- As an owner, I can pre-invite people with a single-use invite code, change anyone's role, revoke access, mute, or kick, at any time.
- As an approved member, I reconnect later and am recognized automatically (cryptographic identity), without asking again.
- As a viewer, I see live updates but my edit attempts are rejected by the **server** (not just greyed out in the UI).

Acceptance criteria

- A raw socket client that skips the UI and sends `OP` or `CHAT` frames without approval is rejected and cannot read state or chat.
- Requests are rate-limited; repeated denied requests are blocked for a cool-down.
- ACL survives server restart.

8. Chat and private messages

User stories

- As a member, I chat with the whole room; each message shows sender, color, and time.
- As a member, I start a **private conversation** with any other member (from the People list or by clicking a name in chat). Only the two of us receive it.
- As a member, if I message someone who is offline, they get it, with an unread badge, when they next connect.

- As a member, when I reconnect or come back the next day, the room chat and my private conversations are still there; I can scroll back through older messages.
- As an owner, I can set how long chat is kept (forever, last N messages, last N days), clear the room chat, and mute a member (mute covers room chat and private messages).

Acceptance criteria

- A third member, or a raw socket client, never receives a private message that is not theirs (CT-02, CT-07).
- Room chat and private messages, with order, timestamps, and unread positions, survive a server restart (CT-08).
- Pending and denied users cannot send or read any chat (CT-05).
- Private messages are never exported in `.lanvas` files and never shown in someone else's Packet Inspector.

Honest limit: private messages are private from other members, not from whoever controls the server machine; they are not end-to-end encrypted (see §9 non-goals).

9. Non-goals

- Internet or cloud sync, accounts, or cloud storage
- Mobile apps
- Full Figma parity (components, auto layout, prototyping, plugins, vector boolean ops)
- Strong anonymity or end-to-end encryption (including of private messages) beyond the LAN transport protections described in `Architecture.md`
- Preventing a viewer with a modified client from capturing a board they are allowed to see

10. Save, recovery, and export (product rules)

- **Ctrl+S always does something useful.** Editors and owners get a verified server checkpoint and a signed local copy; viewers (if allowed to export) get the local copy; if the server is unreachable the local copy is still written and the server checkpoint follows on reconnect.
- **A crash never costs more than the last few hundred milliseconds of edits.** The server repairs itself on restart; the client offers a Recover dialog with unsent changes.
- **Every export opens offline** on another machine and OS: PNG, JPEG, SVG, PDF, `.lanvas`.
- `.lanvas` **files can be trusted:** the app shows who signed a file and warns if it was modified; files can be password-protected.

11. Success criteria

- Demo runs from the downloaded zip on a fresh machine, offline, in under 3 minutes of setup.
- A newly connected device sees the available boards without typing an address.
- 4+ clients draw simultaneously with no divergence (verified by board hash comparison).
- Kill a client's connection mid-stroke; on reconnect the board matches the server.
- Kill the server process; on restart the board, room chat, and private messages are intact.
- An unapproved client can reach the request screen but cannot see or change anything.
- Each team member can explain their module's code and design choices.

12. Demo script (about 8 minutes, rehearse it)

1. Open Client B's launcher first (empty: "Looking for boards..."). Start the server. Client A creates room "Sprint-1" (owner). **Sprint-1 appears on B by itself.**
2. B clicks Sprint-1; sees **Request access** only.

3. Owner approves B as Editor. B's board and recent chat appear instantly (snapshot).
4. Client C joins as Viewer; tries to edit; server rejects.
5. A, B draw and move shapes simultaneously; show cursors, layers, room chat. B sends A a **private message**; C sees nothing.
6. Open the **Packet Inspector**; point out seq numbers and acks; the DM shows as «private» on C.
7. Enable **Chaos mode** (delay and duplicates); show the board stays consistent.
8. Kill B's Wi-Fi; reconnect; show resync from `last_seq`.
9. Undo/redo with two users; show conflict-aware behavior.
10. A presses **Ctrl+S**: "Saved · checkpoint" with matching hash. **Kill the server process**, restart it: board, chat, and DMs are back.
11. Export **PDF** and **PNG**; open them on another machine. Show the signed `.lanvas` and version history (if built).

13. Rubric mapping

Rubric item	Our answer	Doc
Network architecture	Central server, raw TCP sockets, UDP discovery, offline	Architecture §2, §4.4
Protocol design	Framed binary header + JSON, TCP vs UDP justified, chat channels	Architecture §4
User interface	PyQt6 Figma-like UI	Design.md
Performance	Batching, simplification, throttled cursors, batched fsync, stats	Architecture §9
Concurrency	Single-threaded selector loop, global seq, per-channel chat seq	Architecture §5, §4.5
Reliability / recovery	Atomic checksummed storage, integrity scan, client journal	Architecture §12
Security / access	Owner-approved ACL, signed identity, DM routing, signed save files, rate limits	Architecture §8
Extras	Discovery, private chat, export formats, chaos sim, inspector, history	§5 above

14. Risks

Risk	Mitigation
Figma-like scope explodes	Tiering and phase gates; QGraphicsScene gives selection/zoom/pan nearly free; feature freeze at the Phase 7 gate
Demo-day firewall or Wi-Fi isolation (blocks discovery)	Pre-test on router or phone hotspot; server prints IPs; join-by-address fallback; fixed ports documented
UDP port sharing when server and client run on one machine	`SO_REUSEADDR`/`SO_REUSEPORT`; DS-05 on both OSes early (Phase 3)
`fsync`/atomic replace behaves differently on Windows	Shared durable writer; SV-03 and XP-06 on both OSes
Qt packaging issues	First PyInstaller build at the Phase 3 gate, not at the end
Team skill gaps in Qt	Pair UI owner with protocol owner; shared

	`common/` freezes in Phase 0

15. Glossary

- **Op:** one atomic change to the board (create, update, delete).
- **Seq:** server-assigned global order number for each accepted op.
- **Channel:** a chat stream: the room channel or a DM channel between two members; each has its own `chat_seq`.
- **DM:** a private message between two members of the same room.
- **Snapshot:** full serialized board state at a given seq (automatic).
- **Checkpoint:** a snapshot made on Ctrl+S, kept until the owner deletes it, restorable.
- **Recovery journal:** client-side file of the last confirmed board plus unsent changes, used after a crash.
- **`.lanvas`:** native, signed (optionally encrypted) board file.
- **ACL:** access control list for a room (identity to role).
- **Identity:** a client keypair; its public-key fingerprint is the user's stable ID.
- **Phase gate:** the set of tests that must pass before the next phase starts (`Agents.md` §11).

---

16. Acceptance test plan (maps the brief to evidence)

Each row must be **demonstrable offline on the demo hardware**. IDs refer to `Architecture.md` §14. "Evidence" is what we screenshot or record for the PDF/PPT.

#	Requirement	Verified by	Pass criteria	Evidence
R1	Freehand draw, erase, color	manual + UI checklist (`Design.md` §13)	Strokes appear for all; eraser removes whole strokes; color changes sync	screenshot
R2	Real-time sync, minimal delay	PF-01, L3	median < 50 ms, p95 < 150 ms on LAN	inspector screenshot
R3	Server validates and distributes	SE-04, OR-01	invalid ops rejected; valid ops reach all	log excerpt
R4	Shared consistent canvas	OR-01, hash check	all hashes equal	status-bar hash on every machine
R5	Clear canvas with permission; save and restore	AC-01, SV-01, SV-11, RC-05	only permitted roles clear; Ctrl+S hash matches; restore matches hash	video clip
R6	Host session; join by IP + room code	L3 step 5	join works from each OS	screenshot
R7	Participant list; join/ leave notices	manual	notices on join, leave, away	screenshot
R8	Usernames; per-user visual distinction	manual	colored cursors, name tags	screenshot
R9	Join/leave without disruption; new joiner gets state	RC-03, L3	existing users unaffected; newcomer sees full board and recent chat	video clip

R10	Operations over sockets; broadcast	FR-*, OR-*	raw TCP/UDP only, no web libs	code review + inspector
R11	Concurrent editing without lost updates	OR-01, OR-05, CH-01	converge under load and chaos	test report
R12	Ordering and reliability (delayed/duplicate/ missing)	OR-02, OR-03, CH-02, RC-02	documented rules hold	chaos demo
R13	Disconnect detection, timeouts, reconnection	RC-01, RC-06	session survives; client resumes	video clip
R14	Group chat with sender and timestamp	CT-01, manual	chronological, correct sender/time	screenshot
R15	State representation, sync, recovery	RC-03, RC-05	snapshot + replay correct	test report
R16	Undo/redo with defined multi-user behavior	UN-01..UN-05	matches `Architecture.md` §5.5	demo with two users
R17	Access control, no unauthorized modification	AC-01..AC-12, SE-*	unapproved users see and change nothing	raw-socket bypass demo
R18	LAN only, no internet	L3 offline run, EX-07	works with internet disabled	demo-day step
R19	Standalone executables	XP-03	launch from the zip on a clean machine	fresh-machine log
R20	Source, PDF, PPT, videos, single zip	`Agents.md` §14 checklist	all present before submission	zip listing
R21	Private messages between users	CT-02..CT-07, CT-11	only the two members receive them	demo step 5 + raw-socket test report
R22	Chat history retained	CT-08..CT-10	room chat and DMs survive reconnect and server restart	video clip
R23	Available boards shown when a device joins the network	DS-01..DS-08	board appears without typing an address; nothing private advertised	screen recording
R24	Export PNG, JPEG, SVG, PDF, `.lanvas`	EX-01..EX-08	every file opens on another OS offline; `.lanvas` hash matches	exported files
R25	Secure save and file recovery	SV-01..SV-13	Ctrl+S verified; crash at any write step recovers; tampering detected	test report + demo step 10

Demo-day checklist

- ☐ Router/hotspot with internet off; machine IPs noted
- ☐ Firewall rules applied (TCP 47801, UDP 47800, both directions) on server and clients
- ☐ Zip downloaded from the submission and unpacked on the demo machine

- [ ] Client launchers open first; server started; the room appears by itself
- [ ] Owner, editor, viewer, and a rejected visitor ready
- [ ] Chaos mode and inspector tested beforehand
- [ ] Backup: second laptop with the same build; signed `.lanvas` of the board; exported PDF; screenshots of the test log
- [ ] Every member knows their module and can answer questions about it

Architecture.md

How LANvas works. Any change to the protocol or data model **must** update this file, bump `PROTOCOL_VERSION`, and add or adjust tests.

Implementation notes (spec gaps filled while building, no wire change from a released version). `HELLO` also carries `create`, `invite` and `last_seq` (§4.2). `ADMIN` has an S→C reply (§4.2). A sixth op kind, `bulk_create`, gives "bulk create" its wire form (§5.2). Server-to-client frames may reach 64 MB (§4.1). Announces carry `require_code` per room (§4.4). Crash-safety code lives once in `common/durable.py` (§12.1).

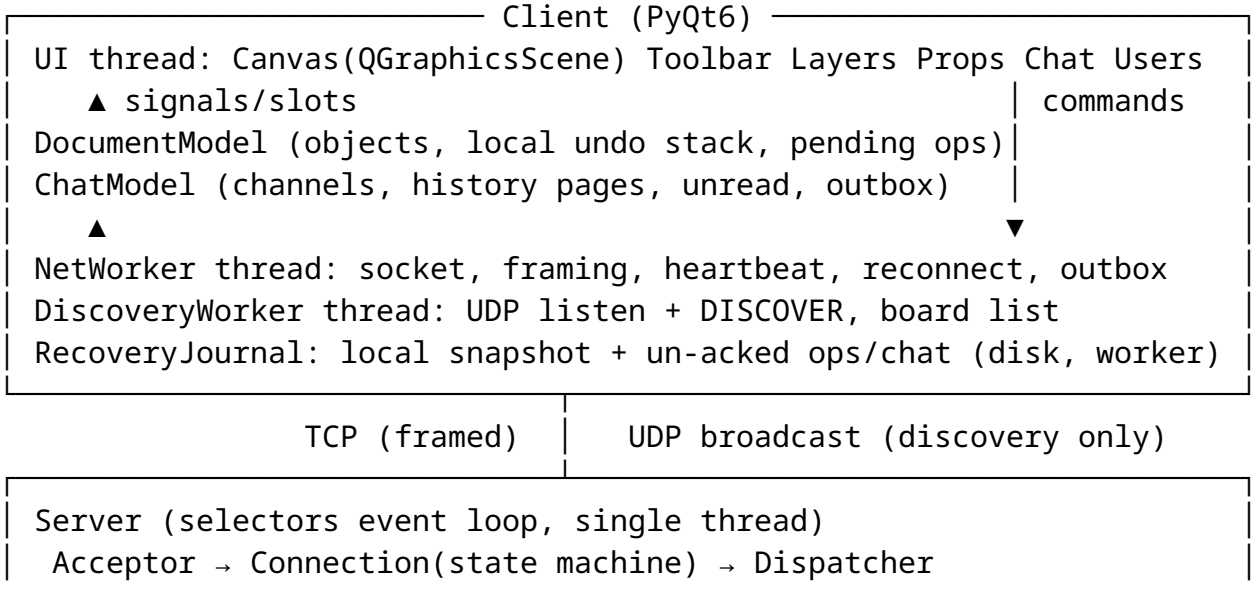
Spec revision 2 adds: private (direct) messages and retained chat history (§4.5), live board discovery (§4.4), secure save with Ctrl+S and file recovery (§12), and export to PNG/JPEG/SVG/PDF/`.lanvas` (§12.5). `PROTOCOL_VERSION = 2`.

---

1. Technology choices

Concern	Choice	Why
Language	Python 3.11+	Required/recommended; fast to build
GUI	**PyQt6** (`QGraphicsScene` / `QGraphicsView`)	Scene graph gives selection, zoom, pan, transforms, z-order
Transport	**TCP** via raw `socket`	Reliable, ordered per connection
Discovery	**UDP** broadcast via raw `socket`	Connectionless, ideal for "who is out there?"
Server I/O	`socket` + `selectors` (single thread)	Low-level, no races, trivial ordering. **No asyncio streams, no socketserver, no websockets**
Crypto	`cryptography` (Ed25519, AES-256-GCM, scrypt, optional TLS cert gen); stdlib `hmac`, `hashlib`, `ssl`	Identity, signed/encrypted save files, transport protection
Export	Qt built-ins: `QImage` (PNG/JPEG), `QSvgGenerator` (SVG), `QPdfWriter` (PDF)	No extra dependency, works offline
Packaging	PyInstaller (build per OS)	Standalone executables
Tests	`pytest`	Protocol, ordering, ACL, chaos, persistence

2. System overview



```
│  AuthManager + ACL │ RoomManager │ OpLog + seq │ Presence          │
│  ChatStore (room + DM channels) │ Discovery(UDP announce/respond)  │
│  Persistence: durable writer (atomic files, checksums, fsync)      │
└────────────────────────────────────────────────────────────────────┘
```

The server is the **single authority** for: membership, permissions, operation order, chat order and retention, and canonical board state. Clients are untrusted.

3. Repository layout

```
lanvas/
├─ common/
│   ├─ protocol.py      # framing, message types, encode/decode, constants
│   ├─ model.py         # object schema, property validation, apply_op()
│   ├─ fractional.py    # fractional indexing for z-order
│   ├─ identity.py      # Ed25519 keys, fingerprints, signatures
│   ├─ chat.py          # chat message schema, channel ids, validation
│   ├─ fileformat.py    # .lanvas read/write: canonical JSON, hash, signature, enc
│   ├─ durable.py       # the durable writer, shared by server and client (§12.1)
│   ├─ netutil.py       # IPv4 interfaces, broadcast addresses, shared discovery so
│   └─ simplify.py      # Ramer-Douglas-Peucker stroke simplification (§9)
├─ server/
│   ├─ main.py          # CLI entry (+ optional status window)
│   ├─ loop.py          # selectors loop, Connection class
│   ├─ rooms.py         # Room, membership, presence
│   ├─ acl.py           # roles, requests, invites, rate limits
│   ├─ oplog.py         # append-only log, seq, snapshots, checkpoints
│   ├─ chatstore.py     # room chat + DM channels, history, retention
│   ├─ discovery.py     # UDP announce/respond
│   ├─ persist.py       # durable writer: atomic replace, fsync, checksums
│   └─ recovery.py      # startup integrity scan and repair
├─ client/
│   ├─ main.py
│   ├─ transport.py     # NetCore: socket thread, handshake, reconnect, inspector,
│   ├─ net.py           # NetWorker: re-emits NetCore events as queued Qt signals
│   ├─ session.py       # one open room: document, chat, journal, saving (no Qt)
│   ├─ boardlist.py     # board tracking, expiry, validation of announces (no Qt)
│   ├─ discovery.py     # DiscoveryWorker (QThread), live board list
│   ├─ document.py      # client-side model, optimistic apply, undo/redo
│   ├─ chatmodel.py     # channels, history paging, unread counts, outbox
│   ├─ recovery.py      # local journal, crash detection, recovery dialog data
│   ├─ export/          # png.py, svg.py, pdf.py, lanvas.py (shared renderer)
│   ├─ canvas/          # view, items, tools (select, pen, shape, text…)
│   ├─ panels/          # layers, properties, chat, users, inspector, access
│   └─ theme.py         # tokens from Design.md
├─ tests/
├─ docs/  (this file, Design.md, ProductContext.md, Agents.md)
└─ build/ (pyinstaller specs)
```

Rules: `common/` and `server/` import **no Qt**. `client/` never imports `server/`. `common/fileformat.py` is shared so that the server and client produce byte-identical canonical JSON and hashes.

4. Protocol

4.1 Framing (TCP is a byte stream)

Every message is one frame:

```
0        2     3     4          5        8
├ magic ─┤ ver ┤type┤  flags   ├ length (u32, big-endian) ─┤ payload ...
  "LV"      u8    u8     u8
```

- Header is 9 bytes: `magic(2) ver(1) type(1) flags(1) length(4)`.
- `flags` bit0 = payload zlib-compressed (used when payload > 1 KB).
- Payload is UTF-8 JSON (human-debuggable, fine for a LAN).
- `length` is capped at 8 MB for client-to-server frames and 64 KB before authentication. Oversize or bad magic closes the connection. Server-to-client frames may reach 64 MB, because a full snapshot with embedded images can exceed 8 MB after compression; inflation is capped at 64 MB in both directions.
- Receiver uses a per-connection buffer and a `recv_exact`-style state machine (non-blocking on the server).

4.2 Message types

Type	Dir	Purpose	Reliability
`HELLO`	C→S	version, username, identity pubkey, and `room_id` (from discovery) **or** `room_code` **or** `create{name, code, discoverable, require_code, show_member_count}`; optional `invite` token and `last_seq`	n/a
`CHALLENGE`	S→C	random nonce for identity proof	n/a
`PROOF`	C→S	Ed25519 signature of nonce	n/a
`ACCESS_PENDING`	S→C	not authorized, request recorded	n/a
`ACCESS_REQUEST`	S→owner	who is asking (name, IP, fingerprint)	logged
`ACCESS_DECISION`	owner→S	approve(role) / deny	logged
`WELCOME`	S→C	role, client_id, `last_seq`, snapshot or delta, `chat_tail`, `dm_index`, `last_checkpoint`	n/a
`OP`	both	board mutation (§5)	sequenced
`OP_ACK` / `OP_REJECT`	S→C	seq assigned, or error code	n/a
`TRANSIENT`	both	cursor, selection, live stroke preview	**not logged**, droppable
`CHAT`	both	room or direct message (§4.5)	sequenced per channel, persisted
`CHAT_ACK` / `CHAT_REJECT`	S→C	`chat_seq` assigned, or error code	n/a
`CHAT_HISTORY`	both	request an older page of a channel / the page in reply	n/a

CHAT_READ	C→S	mark a channel read up to chat_seq (unread sync across devices)	n/a
PRESENCE	S→C	join, leave, away, role change	n/a
RESYNC / SNAPSHOT	both	gap recovery	n/a
SAVE	C→S	Ctrl+S: request a durable server checkpoint (§12.3)	n/a
SAVE_ACK / SAVE_REJECT	S→C	checkpoint seq, board_hash, checkpoint_id, time; or error code	n/a
ADMIN	both	C→S: set_role, kick, revoke, mute, set_settings (includes lock), rename, clear_chat, set_retention, create_invite, list_checkpoints, delete_checkpoint, delete_room. S→C: {action, ok, ...} reply, e.g. the invite token or checkpoint list. A refusal is a non-fatal ERROR	logged
PING / PONG	both	liveness plus RTT	n/a
ERROR	S→C	code and message	n/a

4.3 Why TCP, and why UDP for discovery only

Drawing state and chat need **reliable, ordered** delivery; reimplementing that over UDP adds complexity with no benefit on a LAN. Cursor positions would tolerate loss, but a single TCP connection keeps auth and ordering simple; we mitigate head-of-line blocking by throttling transients. UDP is used only for discovery, which is naturally connectionless and must work before the client knows any address.

4.4 Discovery: live list of available boards (UDP)

Goal: as soon as a device is on the network and the launcher is open, it **shows the boards it can join**, without typing an address.

Server side (`server/discovery.py`**)**

- Binds UDP `47800` with `SO_REUSEADDR` (and `SO_REUSEPORT` where available) and `SO_BROADCAST`.
- **Announces** `LANVAS_ANNOUNCE` every **2 s** to the broadcast address of every IPv4 interface (enumerated, not only `255.255.255.255`), and once immediately on start and whenever a discoverable room is created, renamed, hidden, or deleted.
- **Responds** to `LANVAS_DISCOVER` with a unicast `LANVAS_ANNOUNCE` to the sender.
- Announce payload (JSON, kept under 1200 bytes; paginated with page/pages if a server hosts many rooms):

```
{ "t": "LANVAS_ANNOUNCE", "server_id": "<random per install>", "server_name": "Lab·
  "addrs": ["192.168.1.10"], "tcp_port": 47801, "protocol_version": 2,
  "rooms": [ { "room_id": "r_8f2c…", "name": "Sprint-1", "requires_approval": true
               "require_code": false, "host_name": "Alice", "member_count": 3 } ] }
               # member_count only if the owner allows. require_code tells the laun
```

A server with no rooms still announces (`"rooms": []`), so the Create room dialog can list it. Announces

also go to `127.255.255.255`, so a machine hosting its own server sees it with no network at all.

- **Never advertised:** room codes, member names, board content, chat, hidden rooms.
- Hidden rooms are joinable only by IP + room code.

Client side (`client/discovery.py`, `DiscoveryWorker` thread)

- Opens when the launcher opens (and stays alive in the background while a board is open, for the **Switch board** menu).
- Binds UDP `47800` (shared with a local server via `SO_REUSEADDR`) to receive announces passively, so a device that **just joined the network** sees boards within one announce interval.
- Also sends `LANVAS_DISCOVER` immediately on start, on **Refresh**, and when its set of network interfaces changes (polled every 5 s), so results appear without waiting.
- Maintains a board list keyed by `(server_id, room_id)`; entries **expire after 7 s** without an announce (about 3 missed intervals). The UI is updated by signal only when the list changes.
- Drops packets that are oversized, malformed, from an incompatible `protocol_version` (shown greyed as "needs update"), or that arrive faster than a per-source rate limit.

Joining from the list: the client sends `HELLO{room_id}`. Because the room code is a *locator*, not a credential (§8), a discovered room does not need the code; owner approval or an invite is still required. A room may set `require_code = true`, in which case the launcher asks for the code before sending `HELLO`.

Abuse controls: replies are unicast and never larger than one announce; at most 5 replies/s per source IP; the server ignores discover packets over 512 bytes.

4.5 Chat: room chat, private messages, and retention

Channels

- `room`: one per room, every `ACTIVE` member receives it.
- `dm:<id>`: one per pair of members, `id = sha256(sorted([fp_a, fp_b]))[:16]`. Delivered **only** to the connections of those two fingerprints. Addressing is by **fingerprint**, never username (usernames can collide).

CHAT payload (C→S)

```
CHAT { msg_id: "clientId:counter", channel: "room" | "dm", to: "<fingerprint>" (dm
       text: "≤ 2000 chars", reply_to: <chat_seq>? }
```

Server algorithm (same single thread as ops)

1. Connection must be `ACTIVE`; sender not muted (mute applies to room and DMs); for DMs the recipient must be a current member of the room (online or offline). Otherwise `CHAT_REJECT{code}`.
2. Dedupe by `msg_id` (re-send `CHAT_ACK`).
3. Validate length, control characters stripped, rate limit (token bucket, e.g. 5 msgs/s burst 10).
4. Assign `chat_seq = ++channel.seq`, stamp server UTC time, append to the channel's `.jsonl` file (durable writer, §12.1).
5. Deliver `CHAT{channel_id, chat_seq, from_fp, from_name, ts, text}`: room → all `ACTIVE` members; DM → sender's and recipient's live connections only. Offline DM recipients get it from history on next connect.

Retention (chat is kept, not ephemeral)

- Room chat and DMs are **persisted on the server** and survive client reconnects and server restarts.
- Default retention: **kept for the lifetime of the room**. The owner can set a policy: keep forever / last

N messages / last N days. Pruning runs at startup and every hour; it never removes messages newer than the newest checkpoint export that included chat.

- The owner can **clear room chat** (`ADMIN clear_chat`, logged). Owners cannot read or clear other members' DMs through the UI.
- On `WELCOME`, the client receives `chat_tail` (last 50 room messages) and `dm_index` (for each DM channel: peer fingerprint and name, last message preview, unread count). Older messages are fetched with `CHAT_HISTORY{channel_id, before_chat_seq, limit ≤ 100}`.
- Unread positions are stored server-side per member (`CHAT_READ`) so they follow the user across reconnects.
- Revoked or kicked members stop receiving chat and DMs immediately; their earlier messages stay in history under their name.
- Un-acked outgoing chat is kept in the client outbox and recovery journal and re-sent after reconnect (deduped by `msg_id`).

Honest privacy note (for the report): DMs are hidden from other members, but they are stored in plaintext on the **server machine's disk**, and travel in plaintext on the LAN unless TLS is enabled (§8). They are not end-to-end encrypted.

5. Data model, ordering and concurrency

5.1 Object model (Figma-like)

```
Board
 └─ objects: { id → Object }           # flat map, tree via parent
Object { id, type, parent, z (fractional index), props{...}, deleted }
 type ∈ frame | rect | ellipse | line | arrow | path | text | sticky | image | grou
 props: x, y, w, h, rotation, fill, stroke, stroke_w, opacity, locked,
        points[] (path), text, font_size, … (validated per type)
```

A freehand **stroke is a** `path` **object**; the **eraser deletes path objects** (object-level erase). This avoids pixel-merge conflicts.

5.2 Operations

```
OP { op_id: "clientId:counter", base_seq, kind, ... }
 kind = create {id,type,parent,z,props}
      | update {id,patch:{prop:value,…}}
      | delete {id}
      | clear  {}                       # permission required
      | restore {checkpoint_id}         # owner only: board becomes that checkpoint
      | bulk_create {objects[], replace?}  # undo of clear/restore; replace=true is
```

`restore` is expanded by the server before logging: the op carries the checkpoint's complete object list (tombstones included) and replaces the board in a **single seq**. It is atomic for every client, replay never depends on the checkpoint file still existing, and the board's hash equals the checkpoint's hash exactly (SV-11). Undoing a `restore` is `bulk_create{replace:true}` with the previous objects, which returns the previous hash exactly. Undoing a `clear` is `bulk_create` of the objects it removed. Creating a room from a file uses the same `replace` form.

Fields an op may set are whitelisted by the server. `seq`, `author`, and `author_name` are always the server's own (SE-04).

5.3 Server algorithm (single thread, so no locks)

1. Authenticate connection → role. Reject if role cannot perform `kind`.
2. **Dedupe**: if `op_id` already seen → re-send `OP_ACK` (idempotent).
3. **Validate** schema, property types and ranges, object limits (max objects, max points, max text

length).

4. Assign `seq = ++room.seq`; append to `ops.jsonl` through the durable writer; update canonical state.
5. Broadcast `OP{seq,...}` to all authorized members (including the sender as ack).

5.4 Conflict resolution: per-property last-writer-wins by `seq`

- Two users moving the same object: each property is a register; higher `seq` wins. Everyone converges because the visible state is a pure function of the ordered log.
- `update` on a deleted object is ignored. `delete` wins over concurrent `update`.
- Z-order uses **fractional indexing** (insert between neighbors without renumbering).
- **Optimistic UI:** the client applies its own op immediately, then reconciles to the authoritative `seq` order when the broadcast returns (re-applying still-pending local ops on top).

5.5 Undo / redo (multi-user, well-defined)

- Undo/redo is **per user**: it affects only *your own* actions.
- The client records `(forward, inverse)` for each action. Undo sends the inverse as a **new op** (so history stays append-only and is broadcast normally).
- **Conflict-aware:** an inverse patch applies per property **only if** the current value still equals what your op wrote. Properties changed by others since are skipped, and the UI shows a small toast ("1 change not undone: edited by Sam").
- Undoing a create sends a delete; undoing a delete sends a create with the original id and props.
- `clear` and `restore` are undoable by their issuer via a bulk create of the removed objects.

5.6 Reliability: delayed, duplicated, missing

Problem	Handling
Duplicate op / chat	`op_id` / `msg_id` dedupe on server; `seq` / `chat_seq` dedupe on client
Out-of-order delivery (via chaos shim / reconnect)	Client buffers by `seq` (and per-channel `chat_seq`), applies in order
Gap detected (`seq` jumps)	Client sends `RESYNC{from_seq}`; server replays, or sends `SNAPSHOT` if gap is large
Chat gap	Client requests `CHAT_HISTORY` for the missing range
Lost connection	See §7
Client or server crash	See §12 (durable writes, recovery)

6. Identity, access control and sessions

6.1 Identity

On first launch the client generates an **Ed25519 keypair** (stored in the user's config dir). The **fingerprint** (short hash of the public key) is the stable user ID. Usernames are display-only and may collide; the fingerprint is what the ACL, DM addressing, and save-file signatures track.

6.2 Connection state machine (server)

```
CONNECTED → (HELLO) → CHALLENGED → (PROOF ok) → AUTHENTICATED
   ├─ identity in ACL as member ──────────────► ACTIVE(role)
   ├─ identity pending ────────────────────────► PENDING   (only ACCESS_PENDING / PI
   ├─ identity denied/revoked ─────────────────► DENIED    (close, cool-down)
   └─ unknown identity ──► create request ──► PENDING, notify owner
```

In PENDING/DENIED, the server **never** sends snapshots, ops, chat (room or DM), chat history, checkpoints, or participant lists, and drops any other frame type. This is enforced server-side regardless of the UI.

6.3 Room ACL (persisted in `acl.json`)

```
owner: <fingerprint>
members: { fingerprint → {role, name, added_at, muted} }
requests: { fingerprint → {name, ip, time, status} }
invites: { token_hash → {role, expires, single_use} }
settings: { editors_can_clear, viewers_can_export, require_code, chat_retention, sh
```

Permission matrix:

Action	owner	editor	viewer	pending
Receive board, room chat, presence	✔	✔	✔	✘
create / update / delete objects	✔	✔	✘	✘
Send room chat	✔	✔	✔ (owner can mute)	✘
Send / receive private messages (DMs)	✔	✔	✔ (owner can mute)	✘
Read chat history (room + own DMs only)	✔	✔	✔	✘
Clear room chat, set chat retention	✔	✘	✘	✘
SAVE (server checkpoint, Ctrl+S)	✔	✔	✘	✘
Export / save a local copy (client-side)	✔	✔	owner-configurable	✘
Clear board	✔	owner-configurable	✘	✘
Restore a checkpoint	✔	✘	✘	✘
Approve/deny, set roles, kick, mute, lock board	✔	✘	✘	✘

Role changes and revocation take effect immediately: the server updates the live connection and sends PRESENCE.

Export permission for viewers is enforced by the client UI only: a viewer already receives the full board, so the server cannot stop a modified client from rendering it. The setting is a courtesy control, and the report says so.

6.4 Owner offline

Requests queue in the ACL; owner gets them on next connect. Approved members keep working and chatting. A room may be configured so editors continue even when the owner is away.

7. Connection management and recovery

- PING/PONG every 5 s; no traffic for 15 s → mark **away**; 60 s → release the slot (ACL membership is *not* lost).
- Client reconnects with exponential backoff, presents identity, sends `last_seq` and the last `chat_seq` per open channel.
- Server replies with the ops after `last_seq`, or a full snapshot if too old, plus missed chat. Client keeps un-acked local ops and chat and re-sends them (server dedupes by `op_id` / `msg_id`).
- Server restart: run the integrity scan (§12.4), then rebuild state from the newest valid snapshot plus the `ops.jsonl` tail.
- Automatic snapshots every 200 ops or 2 minutes; manual checkpoints on Ctrl+S (§12.3).

8. Security

- **Authentication:** Ed25519 challenge-response; no passwords on the wire.
- **Authorization:** enforced per frame on the server (see matrix). Clients are never trusted.
- **Room code** is a locator, not a credential. Even with the code, access needs owner approval (or a valid invite).
- **Invites:** random single-use token, stored hashed, with expiry.
- **DM routing:** the server delivers a DM only to the two fingerprints of the channel; history requests are checked against the requester's fingerprint.
- **Save files:** written atomically, checksummed, signed with the saver's identity key, optionally encrypted with a password (§12.5). Files in the user data dir use `0600` where supported.
- **Abuse controls:** frame-size caps, per-connection token-bucket rate limit (ops and chat), per-IP request limits and cool-down, max objects and points, JSON depth limit, discovery reply rate limit.
- **Transport:** Tier 3 target is TLS via `ssl` wrapping the raw socket with a self-signed cert generated on first server run, plus trust-on-first-use pinning in the client. If TLS slips, the access model above still holds, but traffic (including DMs) is plaintext on the LAN. Document this honestly.
- **Threat notes for the report:** same-LAN sniffing (mitigated by TLS), rogue client sending malformed frames (validation), impersonation (identity keys), brute-force requests (rate limits), replay of `PROOF` (fresh nonce per connection), DM snooping by other members (server-side routing), tampered save files (hash + signature), discovery amplification (unicast, rate-limited replies).

9. Performance

- Live pen strokes: client sends `TRANSIENT stroke_preview` batches at about 30 Hz; commits a single `create path` on mouse-up with points simplified by **Ramer–Douglas–Peucker**.
- Drag and resize: send `update` at most every 33 ms with coalescing (only the latest patch per object).
- Cursors/selections: throttled to about 20 Hz, dropped if the outbox is backed up.
- zlib for payloads > 1 KB; snapshots always compressed.
- Server write path uses a per-connection outbox with `EVENT_WRITE` registered only when non-empty; slow clients are disconnected when the outbox exceeds a cap.
- Disk: `fsync` is batched (group commit within a ≤ 100 ms window) so per-op latency stays low; `SAVE` forces an immediate `fsync`.
- Export rendering of large boards is tiled and reports progress; file writing happens off the UI thread.
- Stats (bytes/s, msgs/s, RTT, seq lag) are surfaced in the Packet Inspector.

10. Threading (client)

- **UI thread:** all Qt objects, including the scene render used by export. Never blocks on the network or on large disk writes.
- **NetWorker thread:** owns the TCP socket; receives frames, emits Qt signals (`op_received`, `chat_received`, `presence`, `save_ack`, …); sends from a thread-safe queue.
- **DiscoveryWorker thread:** owns the UDP socket; emits `boards_changed(list)`.
- **IO worker (`QThreadPool`):** recovery journal writes, `.lanvas` save, encoding and writing export files.
- Qt widgets are never touched from worker threads.

11. Packet Inspector and Chaos Simulator (client-side shim)

A `Transport` wrapper sits between framing and the socket:

- **Inspector:** records `(time, dir, type, seq, size, rtt)` into a ring buffer for the UI. DM text is shown as «`private, 42 chars`» unless the inspecting user is a participant of that DM.
- **Chaos:** optionally adds delay, duplicates frames, reorders *application delivery*, or force-closes the connection. Used for tests and the demo.

12. Persistence, secure save, recovery and export

12.1 Durable writer (server and client)

All persistent files go through one helper, `common/durable.py`. `server/persist.py` and `client/recovery.py` re-export it, because two copies of crash-safety code would drift. Three rules:

1. **Whole-file writes are atomic:** write `<name>.tmp` → `flush` → `os.fsync` → `os.replace(tmp, name)` → `fsync` the directory (POSIX). A crash leaves either the old file or the new file, never a half file.
2. **Append-only logs are self-checking:** each line of `ops.jsonl` and chat logs is `{"seq":…, "data":…, "crc":<crc32 of data>}`. A torn or corrupt **last** line is detected and truncated on startup; corruption in the middle stops replay at the last good line and falls back as in §12.4.
3. **Every snapshot and checkpoint carries a SHA-256** of its canonical board JSON, verified before use.

Line endings are normalized to `\n`; all paths use `pathlib`.

12.2 Persistence layout

```
data/rooms/<room_id>/
   room.json                 # name, code hash, settings
   acl.json
   ops.jsonl                 # append-only, one op per line: seq, data, crc
   chat/
      room.jsonl             # room channel, chat_seq-ordered
      dm/<channel_id>.jsonl  # one file per pair of members
      read.json              # unread positions per member
   snapshots/<seq>.json.gz   # automatic, keep the newest 10
   checkpoints/<seq>.json.gz # Ctrl+S checkpoints, kept until the owner deletes the
   checkpoints/index.json    # checkpoint_id, seq, board_hash, saved_by, label, tir
   recovery.log              # what the integrity scan found and fixed
```

Snapshot / checkpoint file:

```
{ "format": "lanvas-snapshot", "version": 2, "room_id": "…", "seq": 1234,
  "created_at": "UTC ISO-8601", "board": { …canonical… }, "board_hash": "sha256:…"
```

12.3 Ctrl+S: secure save

Pressing **Ctrl+S** (or the Save button) does two things, both off the UI thread:

1. **Server checkpoint** (owner/editor): the client sends `SAVE{label?}`. The server flushes and `fsyncs` `ops.jsonl` and chat logs, writes a checksummed checkpoint at the current `seq` atomically, adds it to `checkpoints/index.json`, and replies `SAVE_ACK{checkpoint_id, seq, board_hash, time}`. The client compares `board_hash` with its own; a mismatch triggers `RESYNC` and a warning toast. Viewers get `SAVE_REJECT{NOT_ALLOWED}` and the UI explains it.
2. **Local secure copy** (all roles allowed to export): the client writes a signed `.lanvas` file of the confirmed board into its user data dir (`backups/<room_id>/<timestamp>.lanvas`, newest 20 kept). If the server cannot be reached, this local copy is still written, and the status bar says

"Saved locally; server checkpoint pending". The pending SAVE is sent on reconnect.

Ctrl+Shift+S (Save a copy…) writes a `.lanvas` to a location the user picks, with optional password encryption.

The owner can browse checkpoints (Version history) and `restore` one (§5.2).

12.4 Recovery

Server startup integrity scan (`server/recovery.py`)

1. For each room, load `acl.json` and `room.json`; on parse failure, use `<name>.bak` (the previous atomic version is kept as `.bak`).
2. Pick the newest snapshot or checkpoint whose SHA-256 verifies; skip any that fail and note them in `recovery.log`.
3. Replay `ops.jsonl` lines with `seq` > snapshot seq while CRCs verify; truncate a torn tail.
4. Recompute `board_hash` and log `room, seq, hash, repairs`. The server prints a one-line summary per room.
5. Chat logs get the same CRC check and tail truncation.

Client crash recovery (`client/recovery.py`)

- While a board is open, the client keeps a **recovery journal** in its user data dir: the last confirmed board (rewritten atomically every 30 s or 200 ops) plus an append log of un-acked local ops and unsent chat.
- A `session.lock` file (with PID) is created on open and removed on clean exit.
- On next launch, a stale lock means the client crashed. The **Recover** dialog offers: *Rejoin and send my unsaved changes* (re-sent with original op_ids, so the server dedupes anything that already arrived), *Save recovered board as .lanvas*, or *Discard*.
- If the server's room is gone (server data lost), the owner can create a new room from a `.lanvas` file or a local backup ("Open .lanvas" → new room, owner only).

12.5 File formats and export

All exports are written atomically through the durable writer. Scope can be: whole board, current selection, or one frame.

Format	How	Notes
PNG	`QGraphicsScene.render` into a `QImage`	1x / 2x / 4x scale, transparent or canvas background
JPEG	same, quality 90	no transparency; background filled
SVG	`QSvgGenerator`	vector; text kept as text; images embedded as base64
PDF	`QPdfWriter`	vector; page = fit to content, or A4/A3 landscape; option **each frame as one page**
`.lanvas`	`common/fileformat.py`	native, lossless, re-openable; see below

`.lanvas` **file (version 2)**

```
gzip( canonical JSON {
  "format": "lanvas", "version": 2,
  "room_name": "Sprint-1", "seq": 1234, "saved_at": "UTC ISO-8601",
  "saved_by": { "name": "Alice", "fingerprint": "7F3A-91C2…" , "pubkey": "…" },
  "board": { …canonical… },
  "room_chat": [ … ]?,                        # optional, room channel only; DMs are
```

```
  "board_hash": "sha256:…",
  "signature": "Ed25519 over canonical JSON of every field above except signature"
})
```

- **Integrity:** on open, the client checks `board_hash`, then the signature, and shows "Signed by Alice (7F3A-91C2)" or a warning "File was modified or is unsigned" (opening is still allowed, read-only until the user confirms).
- **Optional encryption** (Save a copy… → "Protect with password"): the gzip payload is encrypted with **AES-256-GCM**; key from **scrypt** (n=2^15, r=8, p=1, random 16-byte salt). Encrypted wrapper: `{"format":"lanvas-enc","version":2,"kdf":"scrypt",…,"salt","nonce","ciphertext"}`. A wrong password or tampering fails the GCM tag check cleanly.
- `.lanvas` is also the format of local backups, recovery exports, and "create room from file".

13. Testing strategy

Test	What it proves
Framing fuzz (split/merged/partial frames)	Stream reassembly
Ordering with N simulated clients	All reach identical board hash
Dedupe and gap/resync	Reliability rules
ACL matrix	Every role/action pair, including raw-socket bypass attempts
Chat routing and retention	DMs reach only their two members; history survives restarts
Discovery	New devices see boards quickly; nothing private is advertised
Undo conflict cases	§5.5 behavior
Reconnect and server restart	Recovery
Crash and corruption injection	Durable writer and integrity scan
Export round trips	Every format opens; `.lanvas` hash survives save/load
Chaos soak	Convergence under delay/duplicates

Convergence check: after any test, every client computes `sha256(canonical_json(board))`; all hashes must match the server's.

---

14. Test procedure (full)

Section 13 is the summary; this is the procedure the team follows. Test IDs are referenced from `ProductContext.md` (acceptance) and `Agents.md` (workflow, phase gates).

14.1 Test layers

Layer	Scope	Needs	Runs
L0 Unit	Pure functions: framing, `apply_op`, fractional index, RDP, identity, ACL rules, chat validation, file format, CRC/atomic writer	nothing	every save / push
L1 Component	Server loop and connection state machine	nothing	every push

	over `socket.socketpair()` and a fake clock		
L2 Integration	Real server on `127.0.0.1:0` (ephemeral port) plus headless **bot clients** over raw TCP and UDP; no Qt	one machine	every push
L3 LAN system	Real GUI clients on **3+ physical machines**, mixed OS, internet disabled	a router/hotspot	at every phase gate that includes a GUI, plus once per working session from Phase 3 on
L4 Acceptance	The demo script and requirement checklist (`ProductContext.md` §12, §16)	whole team	Phase 7 gate and demo day

14.2 Harness (`tests/harness.py`)

- `start_server(tmpdir)` returns `(port, handle)`; uses a temp data dir, so tests never touch real rooms. `handle.kill()` simulates a crash (no clean shutdown); `handle.restart()` runs the integrity scan.
- `BotClient(port, name, key)` implements HELLO/PROOF, `send_op()`, `send_chat()`, `send_dm(fp)`, `save()`, `recv_until(seq)`, `board_hash()`. Bots use the **same** `common/protocol.py` but their own socket code, so framing bugs are not hidden.
- `DiscoveryProbe(port)` sends/receives UDP on a test port.
- `FaultyFS` wraps the durable writer to inject a crash after any write step (after `tmp` write, before `replace`, mid-append).
- `FakeClock` injected into the server for heartbeat, retention, and timeout tests (no real sleeping).
- Randomized tests print their **seed** on failure and accept `--seed` to reproduce.
- UI and export tests use `QT_QPA_PLATFORM=offscreen`; nothing in L0-L2 server tests may need a display.
- pytest markers: `unit`, `integration`, `security`, `chaos`, `perf`, `lan`, `chat`, `discovery`, `persist`, `export`.

14.3 Test case catalogue

Framing (FR)

ID	Case	Pass criteria
FR-01	Frame split at every byte boundary	Reassembled identically
FR-02	Many frames in one `recv`	All decoded, in order
FR-03	Length above cap (8 MB; 64 KB pre-auth)	Connection closed, no memory growth
FR-04	Bad magic or version	Closed with `ERROR`
FR-05	Compressed flag round trip (>1 KB)	Payload equal
FR-06	Invalid JSON, wrong types, JSON depth bomb	Rejected, server alive

Ordering and concurrency (OR)

ID	Case	Pass criteria
OR-01	10 bots x 200 random create/update/delete, concurrent	All client hashes == server hash

OR-02	Inspect assigned `seq`	Strictly increasing, no gaps
OR-03	Same `op_id` sent twice	One `seq`; second returns same ack
OR-04	Update on a deleted object	Ignored, no resurrection
OR-05	Two users set the same property	Higher `seq` wins on all clients
OR-06	1000 z-order inserts in the same gap	No collisions; rebalance works

Undo / redo (UN)

ID	Case	Pass criteria
UN-01	Undo create / redo create	Object removed / restored with same id
UN-02	Undo move (no conflict)	Original position restored
UN-03	Undo move after another user edited that property	Property skipped, toast raised, others still undone
UN-04	User A undo never touches user B's ops	Verified
UN-05	Undo `clear` / undo `restore`	Objects restored by the issuer

Access control and security (AC / SE)

ID	Case	Pass criteria
AC-01	Role x message-type matrix (generated table)	Every cell matches the §6.3 permission matrix
AC-02	Unknown identity connects with correct room code or room_id	Only `ACCESS_PENDING`; **zero** SNAPSHOT/OP/ CHAT/CHAT_HISTORY/PRESENCE bytes
AC-03	Raw `0P` before authentication	Connection closed
AC-04	`PR00F` signed with the wrong key	Denied
AC-05	Replay an old `PR00F` on a new connection	Rejected (fresh nonce)
AC-06	Owner approves a pending connection	Live transition to `ACTIVE`, receives snapshot and chat tail
AC-07	Owner revokes or kicks a live member	Connection downgraded/closed immediately; no further chat or DMs delivered
AC-08	Viewer sends `0P`	`0P_REJECT`; state unchanged on all clients
AC-09	Invite reused / expired	Second use fails; expired fails
AC-10	Access-request flood from one IP	Rate limit plus cool-down
AC-11	Server restart	ACL, roles, pending requests, mute flags preserved
AC-12	Owner offline	Request queues; owner sees it on next login; members keep working
SE-01	10,000 random-byte/garbled frames	Server never crashes; memory bounded
SE-02	Slow-loris (partial frame stalls)	Timeout and close
SE-03	Too many connections from one IP	Cap enforced
SE-04	Oversized arrays/strings/objects in an op	`0P_REJECT` with code

Chat and private messages (CT)

ID	Case	Pass criteria

CT-01	Room message from each role	Delivered to all `ACTIVE` members with server timestamp and correct sender
CT-02	DM from A to B with a third bot C listening on a raw socket	A and B receive it; C receives **zero** bytes of it
CT-03	DM to an offline member	Stored; appears in `dm_index` with unread count on next connect
CT-04	Same `msg_id` sent twice	One `chat_seq`; second returns same ack
CT-05	Pending/denied identity sends CHAT or CHAT_HISTORY, or DMs a member	Dropped; nothing delivered or returned
CT-06	DM to a fingerprint that is not a room member	`CHAT_REJECT{UNKNOWN_RECIPIENT}`
CT-07	`CHAT_HISTORY` for a DM channel the requester is not part of	Rejected, no data
CT-08	Server restart	Room chat and DMs, order and timestamps, and unread positions preserved
CT-09	History paging over 500 messages	Pages are contiguous, no gaps or duplicates
CT-10	Retention "last N" / "N days" (fake clock); owner clears room chat	Only expected messages remain; DMs untouched by room clear
CT-11	Muted member sends room chat / DM	Both rejected
CT-12	Chat flood	Rate limit applies; server responsive

Discovery (DS)

ID	Case	Pass criteria
DS-01	Client starts after server; no user action	Board list populated within 2 s (DISCOVER reply)
DS-02	Client already listening; server starts or a room is created	Board appears within one announce interval (≤ 2 s + jitter)
DS-03	Server stopped / room hidden or deleted	Entry disappears within 7 s
DS-04	Inspect announce bytes	No room codes, member names, board or chat content; hidden rooms absent
DS-05	Multiple interfaces (Wi-Fi + VM bridge) and server and client on the same host	Rooms found on the right interface; port sharing works
DS-06	Malformed, oversized, or flood DISCOVER packets	Ignored; reply rate limited; server responsive
DS-07	Join from the list with `room_id` (no code)	Normal approval flow; `require_code` rooms ask for the code
DS-08	Incompatible `protocol_version` in announce	Shown as not joinable, no crash

Save, persistence and recovery (SV)

ID	Case	Pass criteria
SV-01	Editor sends `SAVE`	`SAVE_ACK` with seq and hash equal to the client's; checkpoint on disk verifies
SV-02	Viewer sends `SAVE`	`SAVE_REJECT{NOT_ALLOWED}`

SV-03	Crash injected at each durable-writer step (`FaultyFS`)	After restart, every file is either the old or the new version; never partial
SV-04	Torn last line in `ops.jsonl` and in a chat log	Truncated, logged in `recovery.log`; hash equals last durable state
SV-05	Newest snapshot corrupted (bad SHA-256)	Falls back to the previous one + replay; hash equals pre-crash
SV-06	`acl.json` corrupted	`.bak` used; ACL intact
SV-07	Client killed with un-acked ops and unsent chat	Recovery dialog on next launch; "Rejoin and send" restores them with no duplicates
SV-08	Server unreachable on Ctrl+S	Local `.lanvas` written; pending `SAVE` sent after reconnect
SV-09	`.lanvas` byte flipped	Open shows "modified or unsigned" warning
SV-10	Encrypted `.lanvas` with wrong password / tampered ciphertext	Clean error, nothing loaded
SV-11	Owner restores a checkpoint, then undoes	Board equals checkpoint hash on all clients; undo returns previous hash
SV-12	Create a room from a `.lanvas` file	New room's board hash equals the file's `board_hash`
SV-13	Local backup rotation	Newest 20 kept per room; older removed

Export (EX)

ID	Case	Pass criteria
EX-01	PNG 1x/2x/4x, transparent and filled	Pixel size matches bounds x scale; alpha as chosen
EX-02	JPEG	Opens; background filled
EX-03	SVG	Valid XML; opens in a browser offline; text selectable
EX-04	PDF fit-to-content and "each frame as page"	Opens; page count equals frame count; vector (not one big image)
EX-05	`.lanvas` save → open	`board_hash` identical; signature verifies
EX-06	Scope: selection / single frame	Only those objects in the output
EX-07	Export with internet disabled, bundled fonts only	Text renders with Inter, no network access
EX-08	Export a 1000-object board	Completes with progress shown; UI responsive (no freeze > 100 ms)

Recovery (RC)

ID	Case	Pass criteria
RC-01	Kill TCP mid-stroke, reconnect	Resumes; no duplicate or lost op
RC-02	Reconnect with `last_seq` inside the replay window	Delta only
RC-03	Reconnect beyond the window	Full snapshot, hash matches
RC-04	Un-acked local ops and chat after reconnect	Re-sent, deduped by `op_id` / `msg_id`

RC-05	Server restart (snapshot + log tail)	Hash equals pre-restart
RC-06	Heartbeat: silent 15 s / 60 s (fake clock)	Marked away / slot released; membership kept
RC-07	One slow client with a full outbox	That client dropped; others unaffected

Chaos (CH)

ID	Case	Pass criteria
CH-01	300 ms delay + 20% duplicates, 5 clients, 60 s, with chat traffic	All hashes equal; chat order identical on all clients
CH-02	Reorder application delivery	Client buffers by `seq`; converges
CH-03	Random disconnects every 5-15 s	Converges; no crash

Performance (PF)

ID	Case	Target
PF-01	Op broadcast latency, 5 clients on LAN	median < 50 ms, p95 < 150 ms
PF-02	300-point pen stroke after RDP	at least 60% fewer points; commit op < 4 KB
PF-03	Load a 1000-object snapshot	< 1 s
PF-04	Pan/zoom with 1000 objects	at least 30 fps
PF-05	Server CPU with 5 idle clients (incl. discovery announces)	< 5%
PF-06	Batched fsync under 5 drawing clients	PF-01 still met

Cross-platform (XP)

ID	Case	Pass criteria
XP-01	L2 suite on Windows and on at least two Linux distros	Green
XP-02	Server on OS A, clients on OS B and C (L3)	Full demo script passes
XP-03	Built executables on each OS (not just `python -m`)	Launch, discover, join, draw, save, export
XP-04	Firewall prompt/rule handled (Windows Defender, `ufw`/`firewalld`) for TCP 47801 and UDP 47800	Documented fix works; discovery works both ways
XP-05	HiDPI (100/150/200%) and Wayland/X11	UI usable
XP-06	Atomic replace and `fsync` on Windows (NTFS) and Linux (ext4)	SV-03 green on both

14.4 Convergence check (used everywhere)

`board_hash = sha256(canonical_json(sorted objects with props, z, deleted flags))`. After any L2/L3/chaos test, every client computes it and all must equal the server's. The same function produces `.lanvas` and checkpoint hashes.

14.5 When tests run

Moment	What
Before every push	`python scripts/check.py` (ruff + L0 + L1 + L2 fast set)
Before merge to `main`	Full L0-L2 including `security`, `chat`, `persist`, and `chaos`

Every phase gate (`Agents.md` §11)	That phase's gate IDs **plus every earlier phase's gate IDs** (full regression); L3 run if the phase has GUI work
Phase 3 onward, each working session	L3 LAN test (14.6) with 3+ machines, mixed OS
Phase 7 gate (release)	First-build-of-record and XP-03 on every OS, full perf, acceptance (`ProductContext.md` §16), fresh-machine unzip test offline
Demo day	Hardware, router, firewall, and demo script dry run

14.6 L3 LAN test script (about 20 minutes)

1. Disconnect all machines from the internet; join the same router/hotspot.
2. Open the client launcher on machines C and D **before** starting the server.
3. Machine A (any OS): start the server; note the printed IP and integrity-scan summary.
4. Machine B: create a room as owner. C and D should see it appear in their launcher **without refreshing** (DS-02).
5. C, D: click the board; request access; one is approved as editor, one as viewer; verify AC-02/AC-08 by hand.
6. B, C draw and move simultaneously for 2 minutes; room chat; C sends a DM to B and D confirms it never sees it (CT-02); undo/redo on both.
7. Unplug C's network for 20 seconds while drawing and chatting, then reconnect (RC-01, RC-04).
8. Enable chaos mode on D for 1 minute (CH-01).
9. Compare the board-hash readout (status bar, `Ctrl+Shift+H`) on all machines: **all must match**.
10. B presses Ctrl+S; check the `SAVE_ACK` hash in the inspector equals every client's hash (SV-01).
11. Kill the server process (no clean shutdown); restart; hashes, room chat and DMs must match (RC-05, CT-08).
12. Force-quit C's client; relaunch; use the Recover dialog (SV-07).
13. Export PNG, SVG, PDF, `.lanvas` on one machine and open each file on another OS (EX-01..EX-05).
14. Record any failure with the bug template (14.8).

14.7 Exit criteria

Phase gates are defined in `Agents.md` §11. In terms of tiers:

- **Tier 1 done (end of Phase 4):** L0-L2 green; FR-*, OR-01..OR-05, AC-01..AC-12, SE-*, CT-01..CT-12, DS-01..DS-08, SV-01..SV-10, RC-01..RC-07, UN-01..UN-04, EX-01, EX-05 pass; one clean L3 run on 3+ machines.
- **Tier 2 done (end of Phase 5):** above plus UN-05, OR-06, SV-11..SV-13, EX-02..EX-08, CH-01..CH-03, PF-01..PF-06, XP-01..XP-03.
- **Release candidate (Phase 7):** all tests green on all demo OSes; XP-04..XP-06; fresh-machine unzip test passes offline.

14.8 Bug handling

Severity	Meaning	Response
S1	Data divergence, data loss, auth bypass, DM leak, crash	Stop-the-line; fix and add a regression test before any other work
S2	Feature broken, no workaround	Fix before the current phase gate can pass
S3	Wrong but with workaround	Schedule within the current or next phase
S4	Cosmetic	Backlog

Bug template: title, severity, OS/distro, build/commit, phase, steps, expected, actual, logs/pcap, seed.
Every fixed bug gets a regression test.

Design.md

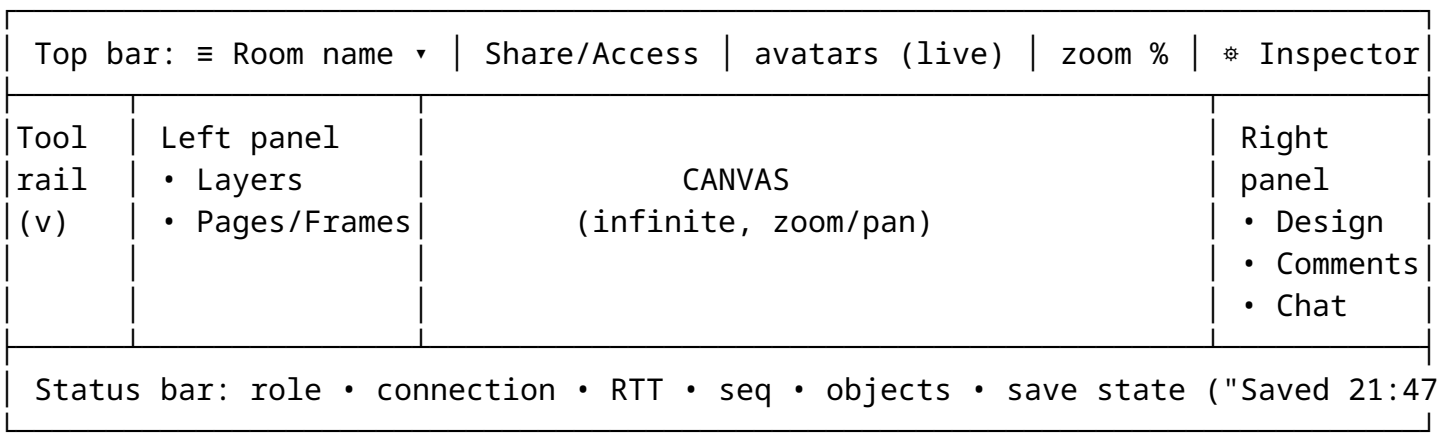
UI/UX design system for LANvas (PyQt6). Goal: **feels like Figma**: calm, dense, tool-focused, with the canvas as the hero. Implement tokens in `client/theme.py`; style with Qt stylesheets (QSS) and custom-painted widgets where needed.

---

1. Design principles

1. **Canvas first.** Chrome is quiet; the board gets the pixels.
2. **Direct manipulation.** Select, drag, resize, rotate on canvas; panels refine.
3. **Presence is visible.** Other people's cursors, selections, and edits are always legible.
4. **Permission is visible.** Your role is always shown; disabled actions explain *why*.
5. **Privacy is visible.** Private conversations always say who can read them.
6. **Saved state is visible.** You can always tell whether your work is safe.
7. **Keyboard-friendly.** Every tool has a single-key shortcut, as in Figma.

2. Window layout



- Panels are collapsible (`Ctrl+\` toggles all UI, as in Figma).
- Floating **bottom-center toolbar** is optional; default is the vertical left tool rail for simplicity of implementation.
- The **≡ Room name ▾** menu contains: Save (`Ctrl+S`), Save a copy… (`Ctrl+Shift+S`), Open `.lanvas`…, Export… (`Ctrl+Shift+E`), Version history, **Switch board ▸** (live list of other boards on the network), Leave room.

3. Tokens

3.1 Color (light and dark)

Token	Light	Dark	Use
`bg.app`	`#F5F5F5`	`#1E1E1E`	window
`bg.panel`	`#FFFFFF`	`#2C2C2C`	panels
`bg.canvas`	`#E5E5E5`	`#1A1A1A`	canvas backdrop
`border`	`#E6E6E6`	`#3C3C3C`	dividers
`text.primary`	`#1E1E1E`	`#F5F5F5`	text
`text.muted`	`#6B6B6B`	`#A0A0A0`	labels
`accent`	`#0D99FF`	`#0D99FF`	selection, focus, primary buttons

`success`	`#14AE5C`	`#14AE5C`	connected, approved, saved
`warn`	`#FFA629`	`#FFA629`	reconnecting, pending, unsaved
`danger`	`#F24822`	`#F24822`	disconnected, delete, deny, save failed
`private`	`#9747FF`	`#B98AFF`	private-message lock icon and DM header strip

User colors (assigned per participant, high contrast, colorblind-aware order): `#F24822` `#0D99FF` `#14AE5C` `#9747FF` `#FFA629` `#FF4DA6` `#00B8A9` `#7A5C00`. Each user color is used for their cursor, name tag, selection outline, and chat name.

3.2 Typography

- UI font: **Inter** (bundled), fallback `Segoe UI`, `SF Pro`, `Noto Sans`. Exports embed or use the bundled font only.
- Sizes: 11 px (labels), 12 px (body/panels), 13 px (headers), 11 px semi-bold (section titles, uppercase).

3.3 Spacing, shape, elevation

- 4 px base grid (4/8/12/16/24).
- Radius: 4 px controls, 6 px popovers, 2 px selection handles.
- Panel width: left 240 px, right 280 px (resizable, min 200).
- Popovers: 1 px border plus soft shadow (`0 4px 16px rgba(0,0,0,.18)`).

3.4 Iconography

Monochrome line icons (Lucide or Phosphor SVG set, bundled), 16 px, 1.5 px stroke. Use `QSvgRenderer`. Lock icon marks private conversations; shield-check marks signed files.

4. Tools and shortcuts

Tool	Key	Notes
Move / Select	`V`	click, shift-click, marquee; handles to resize/rotate
Frame	`F`	container that clips and groups
Rectangle	`R`	`Shift` = square
Ellipse	`O`	`Shift` = circle
Line / Arrow	`L` / `Shift+L`	`Shift` snaps to 15°
Pen (freehand)	`P`	creates `path`; smoothing
Eraser	`E`	deletes whole strokes under cursor
Text	`T`	click to type; double-click to edit
Sticky	`S`	
Comment	`C`	pin comment, threads
Hand	`Space` (hold)	pan
Zoom	`Ctrl+scroll`, `Ctrl+0` (100%), `Shift+1` (fit)	
Undo / Redo	`Ctrl+Z` / `Ctrl+Shift+Z`	per-user
Duplicate / Group / Delete	`Ctrl+D` / `Ctrl+G` / `Del`	

Bring forward / back	`Ctrl+]` / `Ctrl+[`	
Align	`Alt+A/D/W/S/H/V`	left/right/top/bottom/centers
Save	`Ctrl+S`	server checkpoint + signed local copy
Save a copy…	`Ctrl+Shift+S`	`.lanvas` to a chosen location, optional password
Open `.lanvas`	`Ctrl+O`	view, or create a room from it (owner)
Export…	`Ctrl+Shift+E`	PNG, JPEG, SVG, PDF, `.lanvas`
Chat / new private message	`Ctrl+Shift+C` / `Ctrl+Shift+M`	focus chat input / pick a member
Toggle UI	`Ctrl+\`	

5. Canvas behavior

- **Infinite canvas** with smooth zoom (25–400%+) and pan (space-drag, middle mouse, trackpad).
- **Selection:** 1 px accent outline plus 8 resize handles plus rotation zone at corners; multi-select shows a group bounding box.
- **Snap guides:** magenta lines when aligning with neighbors, plus distance labels (stretch).
- **Remote presence:**
  - Cursor arrow in the user's color with a name tag (rounded, white text).
  - Remote selections: 1.5 px outline in that user's color with a small name chip.
  - Live stroke preview appears as the remote user draws.
- **Locked objects** show a lock glyph; moving is blocked.
- **Viewers** get a cursor with an eye icon; edit tools are disabled with tooltip "Viewers can't edit. Ask the owner."

6. Panels

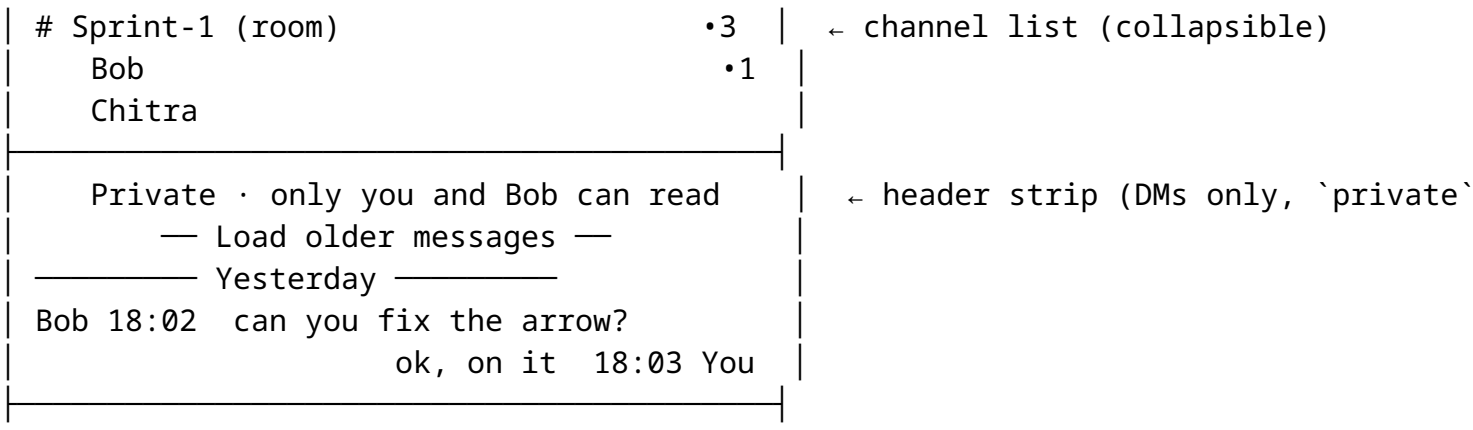
Layers (left)

Tree view with type icon, name, eye (visibility), lock; drag to reorder (writes fractional z); rename on double-click; click selects; colored dot when someone else has it selected.

Design (right)

Contextual sections: Position & size (X, Y, W, H, rotation), Fill, Stroke, Opacity, Text (font size, weight), Alignment buttons, Export (quick "Export selection" shortcut into the Export dialog). Multi-selection shows mixed values as "Mixed". Numeric fields support drag-to-scrub.

Chat (room + private messages)



```
| [ Message Bob…                          ] Send  |
```

- **Channel list** at the top: the room channel first, then private conversations sorted by latest message; each shows an unread badge. The tab badge shows the total unread.
- **Starting a private message:** ✎ New → member picker; or **Message** in the People popover; or click a sender's name in chat → "Message privately". Members are shown by name with a short fingerprint when two names collide.
- **Messages:** grouped by sender with user-color name, timestamp (HH:MM), own messages right-aligned, day separators (Today / Yesterday / date). Scrolling to the top loads older history (spinner, then "Beginning of conversation").
- **Retained history:** the room chat and private conversations are already filled when you join or reconnect; nothing to restore by hand.
- **Offline recipient:** the header shows "Bob is offline; they'll see this when they reconnect."
- **States per message:** sending (faded), sent (normal), failed (red ⟳ retry). Unsent messages survive reconnects and crashes.
- **Roles:** viewers can read and, if not muted, send room messages and private messages. Muted users see the input disabled with "The owner has muted you in this room."
- **Owner extras** (⋯ menu on the room channel): Clear room chat (confirm dialog), Chat retention (Forever / Last N messages / Last N days).

Comments

Pins on canvas; thread list in panel; resolve toggle.

Users and Access

Avatar stack in the top bar (initial letters on user color). Click opens the **People popover**: list with role badges, connection state, **Message** (opens a private conversation), "Following" option, and (owner only) role dropdown, kick, mute.

7. Access-control UX (key flow)

7.1 Visitor: "Request access" screen

Shown when the server returns `ACCESS_PENDING`. Contains: room name, owner name, your fingerprint (short, for verbal verification), a **Request sent, waiting for approval…** state with a subtle spinner, and **Cancel**. No board, chat, or user list is rendered; the board area shows a blurred placeholder illustration, not real data.

States: `Waiting` (amber) → `Approved` (green, auto-transition into the board) / `Denied` (red, with retry cool-down timer).

7.2 Owner: access request toast and dialog

Toast (top right): "**Alice** wants to join Sprint-1" with `Review`. Dialog: name, IP, fingerprint short code (e.g. `7F3A-91C2`), buttons **Approve as Editor**, **Approve as Viewer**, **Deny**, and "Block this device". The owner is encouraged to confirm the fingerprint verbally. Pending requests also appear as a badge on the Share button.

7.3 Share / Access panel (owner)

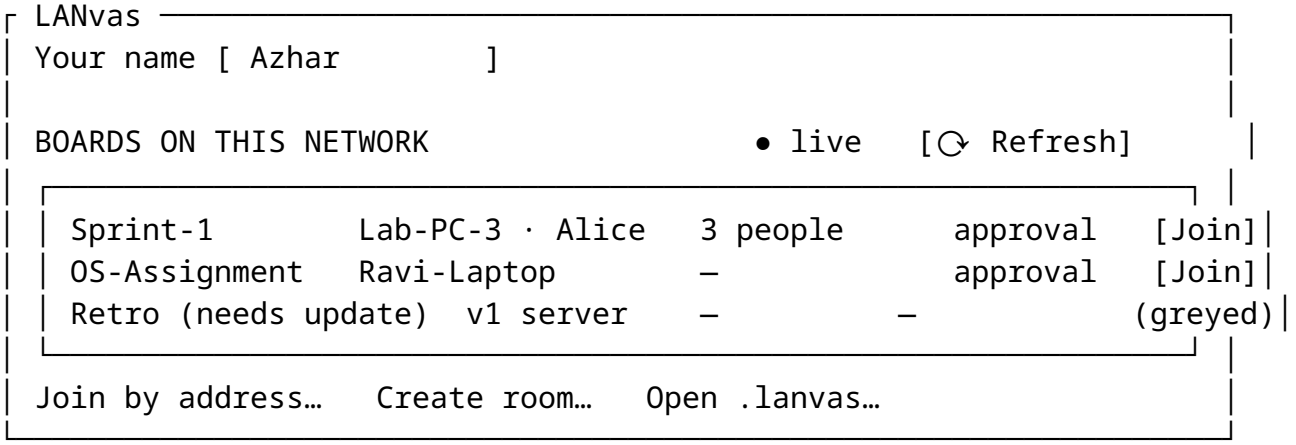
- Room name, room code (copy), LAN address(es) (copy), **Discoverable on this network** toggle (off = hidden), "Require room code to request access", "Show member count in discovery".
- **Create invite** (role, expiry, single-use), shown as a short code.
- Member table: role dropdown, Mute, Revoke, Kick.
- Room settings: lock board (read-only for all but owner), "Editors can clear canvas", "Viewers can export and save copies", "Editors can approve requests" (optional).

7.4 Role indicators

Status bar chip: `OWNER`, `EDITOR`, `VIEWER` with distinct colors. Role changes show a toast ("You are now an editor").

8. Dialogs and flows

8.1 Launcher (first screen): boards on this network



- The list fills **by itself** as soon as the device is on the network: new boards slide in, boards that disappear fade out after a few seconds. A pulsing ● **live** dot shows that listening is active; `Refresh` forces an immediate search.
- Columns: board name, host machine and owner name, member count (only if the owner allows), lock icon meaning "needs owner approval".
- Click a row or **Join** → request flow (§7.1). If the room requires a code, a small code field appears inline first.
- **Empty state:** "Looking for boards on your network..." with spinner; after 10 s, add hints: "Make sure you're on the same Wi-Fi. Allow LANvas through your firewall (UDP 47800). Or **Join by address**."
- **Join by address** (IP, port, room code) remains for hidden rooms and isolated networks.
- Username field remembered locally.
- **Recover banner:** if the last session crashed, a banner at the top opens the Recover dialog (§8.6).

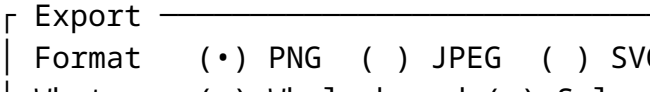
8.2 Create room

Name, code (auto-generated, editable), discoverable toggle (default on), "Start from a `.lanvas` file..." (optional).

8.3 Save (Ctrl+S) and save state

- `Ctrl+S` gives immediate feedback in the status bar: `Saving…` (spinner) → `Saved · 21:47` (green check, tooltip: checkpoint seq, board hash, local copy path) or `Saved locally · server checkpoint pending` (amber) or `Save failed: <reason>` (red, click to retry).
- Between saves, the status bar shows `Edited since last save` in muted text. No modal dialogs on Ctrl+S.
- Viewers without export permission: toast "Viewers can't save copies in this room."
- **Save a copy... (**`Ctrl+Shift+S`**)**: native file dialog plus options: include room chat (off by default; private messages are never included), **Protect with password** (password + confirm, strength hint, "If you forget it, the file cannot be opened").

8.4 Export (`Ctrl+Shift+E`)



```
│ PNG/JPEG scale [1x ▾]   Background (•) Canvas ( ) Transparent │
│ PDF page  [Fit to content ▾]  [ ] Each frame as a page │
│ .lanvas   [ ] Include room chat  [ ] Protect with password │
│ Estimated size: 2400 × 1600 px · ~1.2 MB          │
│                              [Cancel] [Export…]   │
└──────────────────────────────────────────────────┘
```

- Only options relevant to the chosen format are enabled.
- Large exports show a progress bar; the UI stays usable. Success toast: "Exported Sprint-1.pdf · Show in folder".

8.5 Open `.lanvas`

Shows a trust header before anything else: ✓ `Signed by Alice (7F3A-91C2)` (green) or ⚠ `This file was modified or is unsigned` (amber) or a password prompt for protected files. Actions: View (read-only), Create room from this file (owner), Cancel.

8.6 Recover dialog (after a crash)

"LANvas didn't close properly last time. **Sprint-1** has 14 unsaved changes and 1 unsent message." Buttons: **Rejoin and send my changes** (primary), **Save recovered board as .lanvas…**, **Discard**. After rejoining, a toast reports "14 changes recovered."

8.7 Version history (Tier 2 restore, Tier 3 timeline)

List of checkpoints (time, author color, label, hash short code); "Restore this version" (owner, confirm dialog, undoable). Tier 3 adds the timeline slider with author-colored ticks.

9. Network visibility widgets

- **Connection dot** (status bar): green = live, amber = reconnecting, red = offline. Tooltip shows RTT.
- **Packet Inspector** (drawer, `Ctrl+Shift+I`): table columns `Time | Dir | Type | Seq | Size | RTT`, filter by type (incl. `CHAT`, `SAVE`, discovery), pause/resume, row details show decoded JSON (private-message text shown only to its two participants, otherwise «`private`»), and a mini sparkline of msgs/s. A **Discovery** tab lists received announces (source IP, rooms, age).
- **Chaos panel** (inside inspector): sliders for added latency (0–1000 ms) and duplicate probability, buttons "Drop connection" and "Reorder next 5". Clearly marked **Debug**.

10. States and feedback

State	Treatment
Reconnecting	Banner "Reconnecting… your changes are saved locally", editing and chat input stay enabled
Op rejected	Toast with reason ("Viewers can't edit"); local change rolled back
Undo skipped	Toast "1 change not undone: edited by Sam"
Empty board	Subtle hint "Press P to draw, R for a shape"
Large paste/image	Progress indicator, size cap message
Saving / saved / pending / failed	Status-bar save chip (§8.3)
Save hash mismatch	Amber toast "Your view was out of date and has been refreshed", automatic resync
Recovered after crash	Toast "14 changes recovered"
New private message (chat closed)	Toast " Bob: can you fix…" with `Reply`; unread badge

Message failed	Red ⟳ on the bubble; click to retry
Board appeared / disappeared (launcher, Switch board)	Row slides in / fades out; no toast

11. Accessibility and polish

- Minimum contrast 4.5:1 for text; focus ring (2 px accent) on all controls.
- Do not rely on color alone: role badges have text, user cursors have name tags, private conversations have a lock icon and the words "Private", save state has text.
- Scalable UI (respect OS DPI via Qt high-DPI), keyboard navigation of panels, launcher list and dialogs.
- Animations: 120–160 ms ease-out for popovers, toasts, and launcher rows; none on canvas manipulation.

12. Implementation notes (PyQt6)

- Canvas: `QGraphicsView` (+ `setRenderHint(Antialiasing)`, `SmoothPixmapTransform`) with `QGraphicsScene`; custom `QGraphicsItem` subclasses per object type; a `SelectionOverlay` item for handles.
- Remote cursors and selection overlays: a separate layer item, updated from signals, never part of the document, and **hidden during export**.
- Eraser hit-testing: `scene.items(path)` against stroke shapes with a small tolerance.
- Export: one renderer shared by all formats, painting each scene item directly in z-order into `QImage` (PNG/JPEG), `QSvgGenerator` (SVG), or `QPdfWriter` (PDF, one `newPage()` per frame). It is a generator that yields every 50 items, so large exports stay responsive and report progress. File writing runs on the IO worker. Overlays live in `drawForeground`, so they cannot reach an export.
- Chat: a `QTextBrowser` rendering escaped HTML per channel; lazy history loading on scroll-top. Chosen over a `QListView` delegate for grouping and right alignment at far less code; every piece of user text is `html.escape`d first.
- Launcher board list: `QAbstractListModel` fed by the `DiscoveryWorker` signal; diff-based updates so rows animate instead of flickering.
- Dark/light: one Theme dataclass; apply via `QPalette` plus QSS; switch at runtime.
- Keep painting cheap: cache `QPainterPath`, use `ItemCoordinateCache` for heavy items, avoid full-scene updates.

---

13. UI and UX testing

13.1 Manual checklist (run at each phase gate that touches the UI, record in `docs/test-log.md`)

Area	Check	Expected
Tools	Each shortcut in §4 activates the right tool	tool rail highlights
Pen/eraser	Draw, then erase part of a stroke	whole stroke removed for everyone
Select	Click, shift-click, marquee, move, resize, rotate	handles correct; others see live movement
Layers	Reorder by drag, rename, hide, lock	order syncs; locked object cannot move
Properties	Edit X/Y/W/H, fill, stroke, opacity	canvas updates; mixed values show

		"Mixed"
Presence	Two users select/move different objects	colored outlines and name tags appear
Undo/redo	Two users edit the same object, then one undoes	toast appears for skipped change
Launcher	Open launcher, then start a server and create a room on another machine	board appears without refresh; disappears after server stops
Room chat	Send from each role; reconnect; restart server	viewer behavior matches settings; history still there
Private messages	A messages B; C watches	only A and B see it; lock header shown; unread badges correct
Offline DM	Message a member who is offline; they reconnect	message and unread badge appear
Save	`Ctrl+S` as editor, viewer, and with the server unplugged	chip states match §8.3
Recovery	Force-quit the client mid-edit, relaunch	Recover dialog; changes restored once
Access flow	Unapproved user, approve, deny, revoke	screens/states match §7; no board or chat before approval
Network UI	Pull the cable, reconnect	amber banner, editing continues, board recovers
Inspector	Pause/filter/inspect a frame; inspect another user's DM frame	decoded JSON shown, DM text hidden, no UI freeze
Export	PNG/JPEG/SVG/PDF/`.lanvas`, each scope	file opens, matches canvas, no cursors in output
Open file	Open a signed, a modified, and a password-protected `.lanvas`	correct trust header / password prompt

13.2 Visual regression (optional, if time allows)

Render key states offscreen (`QT_QPA_PLATFORM=offscreen`, `QWidget.grab()`): launcher (empty and with boards), request-access screen, board with 3 remote cursors, chat with a DM open, properties panel, export dialog, recover dialog, inspector. Compare to stored baselines with a small pixel tolerance; update baselines only by owner approval.

13.3 Usability test (30 minutes, 2 people outside the team)

Tasks, timed, no help: (1) find a board on the network and get approved, (2) draw a shape and change its color, (3) move an object another user is moving, (4) send a room message and a private message, (5) save and export the board as PDF, (6) find your role. Success: at least 5 of 6 completed unaided. Log every hesitation as a UX bug (S3).

13.4 Display, theme, accessibility

- Check 100%, 150%, 200% display scaling on Windows and a HiDPI Linux machine; X11 and Wayland.
- Light and dark theme: no unreadable text, selection, cursor, and private-message colors remain visible.
- Keyboard-only: every panel, the launcher list, and every dialog reachable; focus ring visible.
- Contrast: spot-check text and badges against the ratios in §11.

13.5 UI performance

Board with 1000 objects: pan/zoom at 30 fps or better; typing in a text object or chat never lags; the

inspector open with a busy stream must not drop canvas frame rate below that (PF-04); exporting the board must not freeze the UI (EX-08).

LANvas user guide

Start

1. **One machine runs the server:** `lanvas-server` (or `python -m server.main`). It prints the LAN addresses to use.
2. **Everyone runs the client:** `lanvas-client` (or `python -m client.main`). Enter your name.
3. **Boards on the network appear by themselves.** Pick one and press **Join**.

If nothing appears after a few seconds, check that you share a network and that the firewall allows UDP 47800, or use **Join by address**.

Create a room

Choose **Create room...**, then name it and pick the server. You become the owner, and the only member.

The room code is shown in **Share** while you stay connected, because the server stores only a hash of it. Note it down if you make a hidden room.

Getting access

Being on the same network is not access. A visitor sees a **Request access** screen with their fingerprint, and nothing from the board.

The owner gets a notice with **Review**, then chooses **Approve as Editor**, **Approve as Viewer**, **Deny**, or **Block this device**. Compare fingerprints out loud if it matters who is asking. Approved members are recognized automatically when they return.

Role	Can do
Owner	Everything, including approvals, roles, restoring versions
Editor	Draw and edit, chat, Ctrl+S
Viewer	Watch and chat; save copies only if the owner allows it

Tools

Key	Tool	Key	Tool
`V`	Select, move, resize, rotate	`P`	Pen
`F`	Frame	`E`	Eraser (removes whole strokes)
`R`	Rectangle	`T`	Text
`O`	Ellipse	`S`	Sticky note
`L` / `Shift+L`	Line / arrow	`C`	Comment

Hold `Space` to pan. `Ctrl`+scroll zooms; `Ctrl+0` is 100%, `Shift+1` fits the board. `Ctrl+D` duplicates, `Ctrl+G` groups, `Del` deletes, `Ctrl+]` and `Ctrl+[` change stacking order. `Ctrl+\` hides the panels.

Undo

`Ctrl+Z` and `Ctrl+Shift+Z` affect only your own changes. If someone has since changed the same property, that property is left alone and a notice says so.

Chat

The **Chat** tab holds the room channel and your private conversations. Start one with `Ctrl+Shift+M`, from **Message** in the People list, or by clicking a name in chat.

Private messages go only to the two people in them. They are not end-to-end encrypted: whoever controls the server machine can read them on its disk, and they cross the network in plaintext.

History stays through reconnects and server restarts. Offline members get their messages, with an

unread badge, when they return.

Saving

Ctrl+S makes a verified checkpoint on the server and a signed copy on your machine. The status bar shows the result. If the server is unreachable, the local copy is still written and the checkpoint follows on reconnect.

Ctrl+Shift+S saves a `.lanvas` wherever you choose, optionally with a password. If you forget the password, the file cannot be opened.

Ctrl+O opens a `.lanvas`, showing who signed it or warning that it was modified. From there you can make a new room from it.

The owner can restore any saved version under **Version history**. Restoring affects everyone, and `Ctrl+Z` undoes it.

Export

Ctrl+Shift+E exports PNG, JPEG, SVG, PDF, or `.lanvas`, for the whole board, the selection, or one frame. PDF can put each frame on its own page.

When things go wrong

- **Connection lost:** keep working. Your changes and messages are sent when the connection returns.
- **The app crashed:** the launcher shows a banner. **Recover** rejoins and sends your unsaved changes, without creating duplicates.
- **The server crashed:** restart it. It repairs itself on startup and prints a line per room. Boards, chat, and private messages come back.

The Packet Inspector

`Ctrl+Shift+I` shows every frame with its sequence number, size, and round-trip time. The **Chaos** tab adds delay and duplicates and drops the connection, to show that the board stays consistent. It affects only your own client.

`Ctrl+Shift+H` shows the board hash. When every client shows the same hash, every client has the same board.

Test log

One row per gate run (`Agents.md` section 11). IDs are from `Architecture.md` section 14.

Date	Phase	Commit	Machines / OS	IDs run	Result	Bugs opened
2026-11-05	0-5 (automated part)	none yet; the repository has no git history	1 machine: Fedora 43, Linux 7.2, Python 3.14.8, PyQt6 6.10.2, loopback only, Qt offscreen	91 IDs from `scripts/gate.py --phase 5`, with `LANVAS_FULL_SOAK=1`	All 91 pass; none fail, are missing, or skip. **Gate NOT GREEN:** 13 items still need other machines or people (below)	none open

Honest status

No phase gate is green yet, and no `phase-N-green` **tag is justified.** Each gate also requires things that one Linux machine cannot show. Everything an automated run on this machine can prove has passed.

Still needed from the team

Item	Needs	Gate
`check.py` and XP-06 on Windows/NTFS	a Windows machine	0
XP-01	full L2 suite on Windows and a second Linux distro	5
XP-02, first L3 run, Tier 1 and Tier 2 exit L3 runs	3+ physical machines, mixed OS, no internet (`Architecture.md` 14.6)	3, 4, 5
XP-03 on every other demo OS	run `python -m build.make` and the `--smoke` check there	3, 5
XP-04	try the firewall commands in `README.md` on real Windows and Linux hosts	3
PF-04 on real hardware	real GPU and display; the 102 fps here is software-rendered and offscreen	5
`Design.md` 13.1 manual checklist, 13.3 usability test, 13.4 HiDPI	people and real displays	4, 5
Tier 1 four-machine LAN test	required by `ProductContext.md` section 6 before any Phase 6 work	6

Measured on this machine (loopback, so not LAN figures)

ID	Target	Result
PF-01 op broadcast, 5 clients	median < 50 ms, p95 < 150 ms	median 1.6 ms, p95 2.0 ms
PF-03 load 1000 objects	< 1 s	9 ms (without painting)
PF-04 pan/zoom, 1000 objects	at least 30 fps	102 fps (software, offscreen)
PF-05 server idle CPU, 5 clients	< 5%	0.3%
PF-06 fsync under load	PF-01 still met	median 1.6 ms, p95 2.8 ms
CH-01 chaos soak	hashes equal	passed at the full 300 ms, 20% duplicates, 60 s

Real networks will be slower than loopback. PF-01 has to be measured again in the L3 run.

How some IDs were tested

- **CH-03** drops connections far more often than the catalogue's 5-15 s, over a shorter run.
- **DS-05** sends one real broadcast between sockets sharing a port. Several real interfaces need the L3 run.
- **XP-03** runs the packaged binaries with no Python environment, but offscreen, and on this OS only.
- **Screens** in `docs/screenshots/` are offscreen renders against a real server process, with other participants played by scripted bots. They are not photos of a multi-machine session. Retake them on real machines for the PPT.

Not built

Phase 6 (Tier 3 and Signature) was skipped, as `ProductContext.md` section 6 requires until Tier 1 passes a 4-machine LAN test.

Smaller gaps against `Design.md`, none in a gate's automated IDs:

- Launcher rows appear and disappear immediately; there is no slide or fade animation (8.1).
- Dark mode is a `--dark` launch flag, with no switch in the UI (12).
- No ungroup command, and the "Following" people option is Tier 3.
- Snap guides have no distance labels (5; marked stretch in the spec).
- The optional "Editors can approve requests" setting is not built (7.3).
- A viewer's cursor is marked by the words "(viewing)" rather than an eye icon (5).
- There is no separate Pages/Frames section in the left panel (2); frames are rows in Layers.

Open question for the team

`ProductContext.md` section 12, step 6 says a DM "shows as «private» on C" in the Packet Inspector. In fact C's inspector shows nothing at all, because the server never sends C that frame (CT-02). That is the stronger property, so the code keeps it. The demo script's wording should change. The «`private`» scrubbing exists in `client/transport.py` and is tested, but with server-side routing it has nothing to act on.

Bugs found and fixed during this run

Each has a regression test, unless the fix is itself in a test.

Found by	Bug	Severity
SV-10	Encrypted files were not recognized: canonical JSON sorts `ciphertext` first, past the marker search	S2
SE-04	A wrong-type string was reported as `LIMIT_EXCEEDED` rather than `BAD_SCHEMA`	S3
Reading the code	`validate_op` refused `z` and `parent` in update patches, which would have broken every reorder	S2
Review	Joining an unreachable address retried forever with no feedback	S2
Review	Leaving a room while offline deleted the journal holding unsent work (data loss)	S1
Looking at renders	Share badge and request toast stayed stale after approval on another device	S3
Looking at renders	Toasts stacked without limit over the panels	S3

Looking at renders	Lock emoji rendered as nothing without an emoji font; several other glyphs were missing from Inter	S3
Looking at renders	White text on the dark private strip was below 4.5:1	S4

Latest gate report

```
# Gate report: phases 0..5

## Phase 0
- FR-01: PASS (2)
- FR-02: PASS (1)
- FR-03: PASS (3)
- FR-04: PASS (3)
- FR-05: PASS (2)
- FR-06: PASS (11)
- SV-03: PASS (15)
- XP-06: PENDING (8 automated pass; needs: also needs a run on Windows/NTFS (Linux
- manual: PENDING (check.py on Windows)

## Phase 1
- OR-01: PASS (1)
- OR-02: PASS (1)
- OR-03: PASS (1)
- OR-04: PASS (2)
- OR-05: PASS (2)
- OR-06: PASS (2)
- RC-02: PASS (1)
- RC-03: PASS (3)
- RC-05: PASS (3)
- RC-06: PASS (1)
- RC-07: PASS (1)
- CT-01: PASS (3)
- CT-02: PASS (1)
- CT-04: PASS (1)
- CT-06: PASS (2)
- CT-07: PASS (7)
- CT-08: PASS (2)
- CT-09: PASS (1)
- CT-10: PASS (7)
- CT-12: PASS (1)
- SV-01: PASS (3)
- SV-03: PASS (15)
- SV-04: PASS (1)
- SV-05: PASS (2)
- SV-06: PASS (1)
- SE-01: PASS (2)
- SE-02: PASS (2)
- SE-03: PASS (1)
- SE-04: PASS (18)

## Phase 2
```

- AC-01: PASS (38)
- AC-02: PASS (3)
- AC-03: PASS (2)
- AC-04: PASS (1)
- AC-05: PASS (1)
- AC-06: PASS (4)
- AC-07: PASS (5)
- AC-08: PASS (1)
- AC-09: PASS (3)
- AC-10: PASS (2)
- AC-11: PASS (1)
- AC-12: PASS (2)
- CT-05: PASS (2)
- CT-11: PASS (1)
- SV-02: PASS (1)

Phase 3
- DS-01: PASS (2)
- DS-02: PASS (2)
- DS-03: PASS (3)
- DS-04: PASS (3)
- DS-05: PASS (3)
- DS-06: PASS (8)
- DS-07: PASS (3)
- DS-08: PASS (2)
- RC-01: PASS (1)
- XP-03: PENDING (1 automated pass; needs: built executables launched on every othe
- XP-04: PENDING (needs: firewall rules verified on Windows and Linux)
- manual: PENDING (L3 LAN run (first))

Phase 4
- UN-01: PASS (1)
- UN-02: PASS (2)
- UN-03: PASS (2)
- UN-04: PASS (1)
- RC-04: PASS (1)
- CT-03: PASS (2)
- SV-07: PASS (4)
- SV-08: PASS (1)
- SV-09: PASS (6)
- SV-10: PASS (3)
- EX-01: PASS (7)
- EX-05: PASS (3)
- manual: PENDING (Design.md 13.1 Tier 1 rows)
- manual: PENDING (L3 LAN run on 3+ machines (Tier 1 exit))

Phase 5
- UN-05: PASS (1)
- SV-11: PASS (2)
- SV-12: PASS (2)

- SV-13: PASS (1)
- EX-02: PASS (1)
- EX-03: PASS (2)
- EX-04: PASS (3)
- EX-05: PASS (3)
- EX-06: PASS (2)
- EX-07: PASS (2)
- EX-08: PASS (1)
- CH-01: PASS (1)
- CH-02: PASS (2)
- CH-03: PASS (1)
- PF-01: PASS (1)
- PF-02: PASS (1)
- PF-03: PASS (1)
- PF-04: PENDING (1 automated pass; needs: frame rate on real GPU and display)
- PF-05: PASS (1)
- PF-06: PASS (1)
- XP-01: PENDING (needs: L2 suite on Windows and a second Linux distro)
- XP-02: PENDING (needs: server and clients on different OSes (L3))
- XP-03: PENDING (1 automated pass; needs: built executables launched on every othe
- manual: PENDING (full Design.md 13.1)
- manual: PENDING (L3 LAN run (Tier 2 exit))

Gate NOT GREEN: 13 item(s) outstanding